# Digital Systems

## Lecture 5

## Finite Width Artithmetic & Branching

Mark van der Wilk

Hilary Term 2024

Department of
COMPUTER
SCIENCE

Following the course of past years by Mike Spivey

**⚠ We have not *precisely* specified finite width arithmetic**

- Registers have limited *width* (32 bits for ARM)
- Can represent $2^{32} = 4294967296$ different values
- Regular arithmetic is specified on a larger set of numbers!

So far, we have implicitly assumed that operations work on **unsigned integers**, i.e. numbers from 0 to $2^{32} - 1$.

Let's *precisely* define:

- how we represent numbers
- what arithmetic operations do

We will illustrate this for 8 bit binary numbers.

**? What happens when we produce a result larger than the max?**

We will precisely specify how condition codes in `psr` are set by arithmetic operations.

**? How do we represent negative numbers?**

**? How can we implement subtraction?**

**? How do branching operations actually work?**

# Representing Numbers: Unsigned

Let's be precise about how bitstrings represent numbers:

$$\begin{aligned} \text{bin} : \{0,1\}^n &\to \mathbb{N} \\ \text{bin}(a) &= \text{bin}(a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0) \\ &= \sum_{i=0}^{n-1} 2^i a_i \end{aligned}$$

Can prove that:

$$0 \leq \text{bin}(a) \leq 2^n - 1$$

# Defining Unsigned Addition

Representing `add` operation on bitstrings as $\oplus$.

We want:

$$\text{bin}(a \oplus b) = \text{bin}(a) + \text{bin}(b)$$

**We cannot find $\oplus$ that satisfies this!**

Finite width of bitstrings means we cannot represent all answers we want. E.g. 254 + 3 = 257, but cannot represent this with 8 bits!

# Defining Unsigned Addition

Instead, want $\oplus$ so that[1]:

$$\mathrm{bin}(a \oplus b) \equiv (\mathrm{bin}(a) + \mathrm{bin}(b)) \qquad \bmod 2^n$$

- Gives the same result as normal addition if result $< 2^n$.
- Result "wraps" around to zero if $\geq 2^n$.
- Can be implemented as normal binary addition, but ignoring bits $\geq n$.

$$\begin{array}{rr} 1011\ 0110 & \\ \underline{0101\ 1101} & + \\ 1\ 0000\ 0011 & \end{array} \qquad \begin{array}{rr} 182 & \\ \underline{93} & + \\ 256 + & 19 \end{array}$$

[1] $a \equiv b \bmod c$ means that $a - b = kc$, for some $k$ (see modular arithmetic).

# What can go wrong?

**Status bits tell us when we deviate from normal arithmetic**

The status bits `N`, `Z`, `C`, `V` are in `psr`. The instruction `adds` is does “add"ition, and sets the “s"tatus bits.

- `N`: …
- `Z`: …
- `C`: **Carry bit**. Set if the unsigned addition overflowed. I.e., a carry was produced which could not be stored in the register.
- `V`: …

# Representing Negative Numbers

We can introduce a *different* mapping from binary representations to numbers, to represent negative numbers.

**Two's Complement Representation**

$$\text{twoc} : \{0,1\}^n \to \mathbb{N}$$

$$\text{twoc}(a) = \sum_{i=0}^{n-2} 2^i a_i - a_{n-1} 2^{n-1}$$

- Easy to see that it can represent negative numbers.
- But has many other nice properties!
- All modern computers use twoc for representing negative numbers.

# Two's Complement Example

- In the last lecture, we encountered a jump instruction with an offset of `0xFA`.

# Two's Complement Example

- In the last lecture, we encountered a jump instruction with an offset of `0xFA`.
- Decoded into binary, we get

$$\texttt{0xFA} = \texttt{0b1111 1010}$$

# Two's Complement Example

- In the last lecture, we encountered a jump instruction with an offset of `0xFA`.
- Decoded into binary, we get

$$\texttt{0xFA} = \texttt{0b1111 1010}$$

- And $\text{twoc}(\texttt{1111 1010}) = -128 + 122 = -6$

# Binary and Twoc Numbers Compared

- $-2^{n-1} \leq \mathrm{twoc}_n(a) \leq 2^{n-2} - 1$

| *a* | *bin(a)* | *twoc(a)* |
|---|---|---|
| 00000000 | 0 | 0 |
| 00000001 | 1 | 1 |
| 00000010 | 2 | 2 |
| 00000011 | 3 | 3 |
| ... | ... | ... |
| 01111111 | 127 | 127 |
| 10000000 | 128 | -128 |
| 10000001 | 129 | -127 |
| ... | ... | ... |
| 11111110 | 254 | -2 |
| 11111111 | 255 | -1 |

# Binary and Twoc Numbers Compared

- $-2^{n-1} \leq \mathrm{twoc}_n(a) \leq 2^{n-2} - 1$
- $\mathrm{twoc}_n(a) = \mathrm{bin}_{n-1}(a_{n-2:0}) - a_{n-1}2^{n-1}$

| *a* | *bin(a)* | *twoc(a)* |
|---|---|---|
| 00000000 | 0 | 0 |
| 00000001 | 1 | 1 |
| 00000010 | 2 | 2 |
| 00000011 | 3 | 3 |
| ... | ... | ... |
| 01111111 | 127 | 127 |
| 10000000 | 128 | -128 |
| 10000001 | 129 | -127 |
| ... | ... | ... |
| 11111110 | 254 | -2 |
| 11111111 | 255 | -1 |

# Binary and Twoc Numbers Compared

- $-2^{n-1} \leq \text{twoc}_n(a) \leq 2^{n-2} - 1$
- $\text{twoc}_n(a) = \text{bin}_{n-1}(a_{n-2:0}) - a_{n-1}2^{n-1}$
- So, twoc and bin are equal, or differ by $2^n$

| *a* | *bin(a)* | *twoc(a)* |
|---|---|---|
| 00000000 | 0 | 0 |
| 00000001 | 1 | 1 |
| 00000010 | 2 | 2 |
| 00000011 | 3 | 3 |
| ... | ... | ... |
| 01111111 | 127 | 127 |
| 10000000 | 128 | -128 |
| 10000001 | 129 | -127 |
| ... | ... | ... |
| 11111110 | 254 | -2 |
| 11111111 | 255 | -1 |

# Binary and Twoc Numbers Compared

- $-2^{n-1} \leq \mathrm{twoc}_n(a) \leq 2^{n-2} - 1$
- $\mathrm{twoc}_n(a) = \mathrm{bin}_{n-1}(a_{n-2:0}) - a_{n-1}2^{n-1}$
- So, twoc and bin are equal, or differ by $2^n$

$$\mathrm{twoc}(a) \equiv \mathrm{bin}(a) \qquad \bmod 2^n$$

| *a* | *bin(a)* | *twoc(a)* |
|---|---|---|
| 00000000 | 0 | 0 |
| 00000001 | 1 | 1 |
| 00000010 | 2 | 2 |
| 00000011 | 3 | 3 |
| ... | ... | ... |
| 01111111 | 127 | 127 |
| 10000000 | 128 | -128 |
| 10000001 | 129 | -127 |
| ... | ... | ... |
| 11111110 | 254 | -2 |
| 11111111 | 255 | -1 |

# Binary and Twoc Numbers Compared

- $-2^{n-1} \leq \mathrm{twoc}_n(a) \leq 2^{n-2} - 1$
- $\mathrm{twoc}_n(a) = \mathrm{bin}_{n-1}(a_{n-2:0}) - a_{n-1}2^{n-1}$
- So, twoc and bin are equal, or differ by $2^n$

$$\mathrm{twoc}(a) \equiv \mathrm{bin}(a) \qquad \bmod 2^n$$

- And so, with a bit of work, can show

$$\mathrm{twoc}(a \oplus b) \equiv \mathrm{twoc}(a) + \mathrm{twoc}(b) \bmod 2^n$$

| *a* | *bin(a)* | *twoc(a)* |
|---|---|---|
| 00000000 | 0 | 0 |
| 00000001 | 1 | 1 |
| 00000010 | 2 | 2 |
| 00000011 | 3 | 3 |
| ... | ... | ... |
| 01111111 | 127 | 127 |
| 10000000 | 128 | -128 |
| 10000001 | 129 | -127 |
| ... | ... | ... |
| 11111110 | 254 | -2 |
| 11111111 | 255 | -1 |

**Addition is the same for both signed and unsigned numbers!**

- It's just that the interpretation of the binary string is different!
- Same addition circuit can be used for signed and unsigned numbers!

# Subtraction

**? How can we implement subtraction?**

Since $b - a = b + (-a)$, let's first consider how to negate a number.

## Negation

Take $\overline{a}$ to be the binary complement of $a$, i.e. $\overline{a}_i = 1 - a_i$.

$$\begin{aligned}
\operatorname{twoc}(\overline{a}) &= \sum_{i=0}^{n-2} (1 - a_i) 2^i - (1 - a_{n-1}) 2^{i-1} \\
&= -\operatorname{twoc}(a) + \sum_{i=0}^{n-2} 2^i - 2^{i-1} \\
&= -\operatorname{twoc}(a) - 1
\end{aligned}$$

So, for all bit strings $a$, we have

$$\operatorname{twoc}(\overline{a} \oplus 1) = -\operatorname{twoc}(a) \qquad \bmod 2^n$$

Few things to note:

- Overflow in $\oplus$ makes negation of $0$ correct (stays zero).
- Negation of $-128 = \texttt{0b1000 0000}$, remains itself, which is correct mod $2^n$.
  - Makes sense, since $+128$ cannot be represented in twoc.

# Subtraction

**? How can we implement subtraction?**

Since $b - a = b + (-a)$, let's first consider how to negate a number:

$$\mathrm{twoc}(\overline{a}) = -\mathrm{twoc}(a) - 1.$$

So, this allows us to define $b \ominus a = b \oplus \overline{a} \oplus 1$, where $\oplus\, 1$ can be implemented by starting with a carry of 1!

$$\mathrm{twoc}(b \ominus \overline{a}) = \mathrm{twoc}(b) - \mathrm{twoc}(a) \qquad \bmod 2^n,$$

## What can go wrong?

Overflows can occur in signed arithmatic too: $\text{twoc}(127 \oplus 1) = -128$.

- Overflow is impossible if the signs are opposite.
- If numbers have same signs, but result has opposite sign, then a **signed overflow** has occurred.

Status bits:

- `N`: ...
- `Z`: ...
- `C`: **Carry bit**. Set if the unsigned addition overflowed. I.e., a carry was produced which could not be stored.
- `V`: **Overflow bit**. Set if signed addition overflowed (test the above).

# Branching & Comparisons

# Conditional Branching

- If/else and loops all require branching if a particular condition is true.

# Conditional Branching

- If/else and loops all require branching if a particular condition is true.
- ARM provides many branching instructions to make this convenient.

# Conditional Branching

- If/else and loops all require branching if a particular condition is true.
- ARM provides many branching instructions to make this convenient.
- The condition of a branch is always a property of arithmetic instr.

# Conditional Branching

- If/else and loops all require branching if a particular condition is true.
- ARM provides many branching instructions to make this convenient.
- The condition of a branch is always a property of arithmetic instr.

**In ARM, branching is conditional on state of `psr` status bits**

# Conditional Branching

- If/else and loops all require branching if a particular condition is true.
- ARM provides many branching instructions to make this convenient.
- The condition of a branch is always a property of arithmetic instr.

**In ARM, branching is conditional on state of `psr` status bits**

- Properties of arithmetic outcomes are stored in `psr` **status bits**.

# Conditional Branching

- If/else and loops all require branching if a particular condition is true.
- ARM provides many branching instructions to make this convenient.
- The condition of a branch is always a property of arithmetic instr.

**In ARM, branching is conditional on state of `psr` status bits**

- Properties of arithmetic outcomes are stored in `psr` **status bits**.
- In addition to checks if arithmetic goes wrong, `psr` contains bits helpful for branching.

## Status Bits

Status bits:

- `N`: **Negative bit**. If the result is negative, i.e. MSB is one.
- `Z`: **Zero bit**. If the result is zero, i.e. all bits in arithmetic result are zero.
- `C`: **Carry bit**. Set if the unsigned addition overflowed. I.e., a carry was produced which could not be stored in the register.
- `V`: **Overflow bit**. Set if signed addition overflowed (test the above).

# Branching Instructions: Equality

Branching instructions interpret the `psr` bits as though a subtraction was performed.

Compare instruction `cmp` performs subtraction, sets `psr` status bits, and does not write result to register.[2]

Equality branches:

- `beq`: Branch if equal. Branches if `Z`.
- `bne`: Branch if not equal. Branches if `!Z`.

---

[2] Other instructions (e.g. `adds`, `subs`) also set `psr` status bits, and can also be used to branch.)

# Branching Instructions: Signed Comparison

`blt`: Branch if less than (signed). Can we figure out which condition we need to hold the status bits to?

If $\mathrm{twoc}(a \ominus b) < 0$ then either

- we really have $\mathrm{twoc}(a) < \mathrm{twoc}(b)$, or
- the subtraction overflowed (e.g. if $\mathrm{twoc}(b) < 0 < \mathrm{twoc}(a)$).

So for this, we need `N` and `!V`.

If $\mathrm{twoc}(a \ominus b) \geq 0$ then either

- we really have $\mathrm{twoc}(a) \geq \mathrm{twoc}(b)$, or
- the subtraction overflowed (e.g. if $\mathrm{twoc}(a) < 0 < \mathrm{twoc}(b)$)

So for this, we need `!N` and `V`.

# Branching Instructions: Signed Comparison

So overall, we need (`N` and `!V`) or (`!N` and `V`), which we can summarise as just needing `N != V`.

# Branching Instructions: Unsigned Comparison

`blo`: Branch if lower (unsigned). Can we figure out the condition?

The same $\ominus$ operation performs subtraction on unsigned numbers, modulo $2^n$ (same reasoning as $\oplus$, going from bin to twoc).

Example: $32 - 9$:

$$\begin{array}{r} 0010\ 0000 \\ 1 \\ \underline{1111\ 0110}\ + \\ 1\ 0001\ 0111 \end{array}$$

If $\text{bin}(a) \geq \text{bin}(b)$, then we get an overflow.

# Branching Instructions: Unsigned Comparision

Proof:

$$\begin{aligned}
&\mathrm{bin}\left(\overline{b}\right) + \mathrm{bin}(b) = 2^n - 1 && \text{Every bit in sum is 1.} \\
&\mathrm{bin}\left(\overline{b}\right) + 1 + \mathrm{bin}(a) \geq 2^n && \text{From } \mathrm{bin}(a) \geq \mathrm{bin}(b) \\
&\therefore a \oplus b \oplus 1 \text{ must overflow.}
\end{aligned}$$

Similar argument showing that if $\mathrm{bin}(a) < \mathrm{bin}(b)$, there is no overflow.

So all we need is `!C`.

## Summary

- Full specification of how bit patterns represent numbers, signed (bin) and unsigned (twoc).
- Full specification of addition and subtraction, and the limitation to *modular arithmetic*.
- Full specification of status bits, as signifiers of departure from regular arithmetic, and flags helpful for branches (`Z` and `N` flags).
- Derivation of flags required for various branch operations (for exam, must demonstrate/understand reasoning, but no need for memorisation, see instruction list).